MAIN PROJECT REPORT ON

SMART TRAFFIC MANAGEMENT SYSTEM USING YOLO

Submitted in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

In

Artificial Intelligence and Machine Learning

By

V.V.M. Sri Vaishnavi(22A81A6162)

Abdul Sameer(22A81A6101)

V. Purna Chaitanya(22A81A6165)

M. Siva Nandini(22A81A6139)

K. Manaswini(22A81A6129)

Ch. Chandu(22A81A6114)

Under the Esteemed Supervision of

Mr. V. Jaya Rama Krishna

M.Tech (Ph. D) Assistant Professor



Department of Artificial Intelligence and Machine Learning

SRI VASAVI ENGINEERING COLLEGE(Autonomous)

(Affiliated to JNTUK, Kakinada)

Pedatadepalli, Tadepalligudem-534101, A.P 2025-2026

SRI VASAVI ENGINEERING COLLEGE (Autonomous)

Department Of Artificial Intelligence and Machine Learning
Pedatadepalli, Tadepalligudem

Certificate

This is to certify that the Project Report entitled "**Smart Traffic Management System Using Yolo**" is submitted by **V.V.V. Sri Vaishnavi(22A81A6162), Abdul Sameer (22A81A6101), V. Purna Chaitanya (22A81A6165), M. Siva Nandini (22A81A6139), K. Manaswini (22A81A6129), Ch. Chandu (22A81A6114)** for the award of the degree of Bachelor of Technology in Artificial Intelligence and Machine Learning of Sri Vasavi Engineering College (Autonomous) during 2025-2026.

Name of Project Guide
Mr. V. Jaya Rama Krishna
M.Tech (Ph. D)

Head of the Department
Dr. Loshma Gunisetti
M.Tech, Ph. D,

External Examiner

DECLARATION BY THE STUDENTS

We hereby declare that the project report entitled **"Smart Traffic Management System Using Yolo"** submitted by us to Sri Vasavi Engineering College (Autonomous), Tadepalligudem affiliated to JNTU Kakinada in partial fulfillment of the requirement for the award of the **B. Tech Degree** in **Artificial Intelligence and Machine Learning** is a record of Bonafide project work carried out by us under the guidance of **Mr. V. Jaya Rama Krishna, Assistant Professor.** We further declare that the work reported in this project has not been submitted and will not be submitted either in part or in full, for the award of any other degree in this institute or any other institute on university.

PROJECT ASSOCIATES

V.V.M. Sri Vaishnavi(22A81A6162)
Abdul Sameer (22A81A6101)
V. Purna Chaitanya (22A81A6165)
M. Siva Nandini (22A81A6139)
K. Manaswini (22A81A6129)
Ch. Chandu (22A81A6114)

ACKNOWLEDGEMENT

We wish to take this opportunity to express our deep gratitude to all the people who extended their cooperation in various ways during our project work. This is our pleasure to acknowledge the help to all those individuals.

Our sincere thanks to our guide **Mr. V. Jaya Rama Krishna –Assistant professor** for timely and kind help, guidance and suggestions and ideas during the development of the project.

It's with a lot of happiness that we are expressing gratitude to **Dr. G. Loshma**, Head of the Department of Artificial Intelligence and Machine Learning, for timely and kind help, guidance and suggestions throughout the entire project.

Our heartfelt thanks to **Dr. G.V.N.S.R RATNAKARA RAO**, principal of Sri Vasavi Engineering College, Pedatadepalli, for providing a favorable environment and support in developing this project.

Our Special thanks to management and all the teaching and non-teaching members, Department of Artificial Intelligence and Machine Learning for their support and cooperation during the project report work.

PROJECT ASSOCIATES

V.V.M. Sri Vaishnavi(22A81A6162)
Abdul Sameer (22A81A6101)
V. Purna Chaitanya (22A81A6165)
M. Siva Nandini (22A81A6139)
K. Manaswini (22A81A6129)
Ch. Chandu (22A81A6114)

ABSTRACT

Now adays traffic congestion has become a major concern in our day-to-day activities because of rapid increase in automobiles and also large time delays between traffic signals. Most of the present-day systems use predetermined timing circuits to operate traffic signals. Difficulty also arises when there is a need for the emergency vehicles like ambulance, or fire engine to cross the lane during the peak hours. We need a smart traffic system that adjusts traffic flow based on how many vehicles are on the road to solve this problem Our project aims to optimize the delay time at traffic signals by calculating the density of the vehicles in a particular lane and providing the green signal time according to density of the vehicles. And our projects also aim for detecting of the emergency vehicles and giving priority to that particular lane. The aim of our project is optimizing waiting time, keeping traffic flow smoothly, and ensuring that emergency vehicles can move quickly without having to wait at traffic signals.

TABLE OF CONTENTS

S.NO	TITLE	PAGE NO
Chapter 1	**INTRODUCTION**	**1-4**
1.1	Introduction	2
1.2	Motivation	2
1.3	Scope	3
1.4	Project Outline	3-4
Chapter 2	**LITERATURE SURVEY**	**5-6**
Chapter 3	**SYSTEM STUDY AND ANALYSIS**	**7-10**
3.1	Problem Statement	8
3.2	Existing System	8
3.3	Limitations Of Existing System	8
3.4	Proposed System	9
3.5	Functional Requirements	9
3.6	Non-Functional Requirements	10
3.7.	System Requirements	10
3.7.1	Hardware Requirements	10
3.7.2	Software Requirements	10
Chapter 4	**SYSTEM DESIGN**	**11-15**
4.1	System Architecture Design	12-14
4.2	Use Case Diagram Using UML Approach	15
Chapter 5	**TECHNOLOGIES**	**16-22**
5.1	Python	17
5.2	TensorFlow	18
5.3	PyTorch	19
5.4	HTML	20
5.5	CSS	21

5.6	Yolo	22
Chapter 6	**IMPLEMENTATION**	**23-31**
6.1	Implementation	24-29
6.2	Simulation	30-31
Chapter 7	**TESTING**	**32-34**
7.1	Introduction to Testing	33
7.2	Testing Strategies	33
7.3	Unit Testing	33
7.4	Integration Testing	34
7.5	System Testing	34
7.6	Acceptance Testing	34
7.7	Test Approach	34
7.8	Test cases	34
Chapter 8	**RESULTS AND DISCUSSION**	**35-40**
8.1	Results	36-40
Chapter 9	**CONCLUSION & FUTURE WORK**	**41-43**
9.1	Conclusion	42
9.2	Future Work	43
Chapter 10	**REFERENCES**	**44-45**

Chapter-1

INTRODUCTION

1.1 Introduction:

This project presents an advanced **Smart Traffic Management System** that leverages the **YOLOv8 (You Only Look Once, Version 8)** algorithm for real-time vehicle detection and intelligent signal control. The system is designed to enhance urban traffic efficiency by automating signal operations, minimizing congestion, and ensuring timely emergency responses. At its core, the system utilizes the YOLOv8 object detection algorithm to accurately detect, classify, and count vehicles approaching from multiple lanes in real time. With its high inference speed and precision, the model efficiently identifies various vehicle types such as cars, buses, bikes, and emergency vehicles.

Based on the detected vehicle count, the system dynamically adjusts the duration of green lights for each direction through adaptive signal control, optimizing traffic flow and reducing waiting time during peak hours for smoother movement across intersections. A crucial feature of this system is its ability to recognize emergency vehicles—including ambulances, police cars, and fire trucks—using YOLOv8's advanced detection capabilities. Upon identifying an emergency vehicle, the system automatically overrides the existing traffic sequence to grant immediate passage, thereby ensuring faster emergency response and saving critical time.

1.2 Motivation:

The increasing number of vehicles and poor traffic coordination have led to severe congestion, delays, and accidents in urban areas. Traditional traffic systems fail to adapt to real-time conditions, causing inefficiency and frustration. This project is motivated by the need for an intelligent, automated solution that ensures smoother traffic flow and faster emergency response.

Key Points:

- To reduce traffic congestion through real-time monitoring.
- To improve emergency vehicle movement and response time.
- To minimize human intervention in signal control.
- To enhance overall road safety and transportation efficiency.

1.3 Scope:

The Smart Traffic Management System using YOLOv8 aims to improve urban traffic efficiency by leveraging real-time vehicle detection and adaptive signal control. It provides a foundation for intelligent, automated traffic solutions that can be scaled for smart city applications.

Scope in Points:

- Real-time detection and classification of vehicles on multiple lanes.
- Dynamic adjustment of traffic signal timings based on traffic density.
- Priority management for emergency vehicles like ambulances and fire trucks.
- Integration potential with IoT-enabled traffic lights and smart city infrastructure.
- Collection of traffic data for analysis and future urban planning.
- Basis for future enhancements such as accident detection and automated traffic rule enforcement.

1.4 Project Outline:

1. **Introduction**
    - Overview of traffic congestion problems
    - Need for intelligent traffic systems
    - Role of AI and YOLO in traffic management
2. **Objectives**
    - Real-time vehicle detection and counting
    - Dynamic signal control based on traffic density
    - Emergency vehicle identification and prioritization
3. **Literature Survey**
    - Review of previous systems using image processing or sensors
    - Comparison with YOLO-based approaches
    - Gaps identified in existing research
4. **System Architecture**
    - Input (video feed from cameras)
    - YOLOv8 detection and classification module
    - Signal control logic
    - Output (adaptive signal management)

5. **Implementation**
    - Dataset collection and preprocessing
    - YOLOv8 model training and testing
    - Integration with signal control system
6. **Results and Analysis**
    - Accuracy and performance metrics
    - Comparative analysis with manual systems
    - Efficiency improvements in traffic flow
7. **Future Scope**
    - Integration with IoT and cloud platforms
    - Real-time traffic prediction using AI
    - Implementation in smart cities
8. **Conclusion**
    - Summary of achievements
    - Importance and potential of the system

Chapter-2

LITERATURE SURVEY

2.Literature Survey:

This section reviews various research works related to intelligent and adaptive traffic management systems. The selected studies focus on technologies like deep learning, IoT, and computer vision to enhance traffic control efficiency and reduce congestion.

1. Lucas Koch, Tobias Brinkmann, and Marius Wegener (2023) proposed an *Adaptive Traffic Light Control with Deep Reinforcement Learning* technique to enhance urban traffic flow. The study applies **DRL-based adaptive control**, which dynamically adjusts signal timings based on real-time vehicle density. This approach significantly improves **traffic flow efficiency** and reduces **energy consumption** when compared with conventional fixed or actuated signal systems. The results demonstrated a more sustainable and intelligent method for traffic management.

2. UmeshKumar Lilhore and Subhendu Kumar Pani (2022) developed an *ML and IoT-based Adaptive Traffic Management System for Smart Cities*. Their system integrates **machine learning algorithms** with **IoT-enabled sensors** to collect and process live traffic data from multiple intersections. Using this data, it makes intelligent, automated decisions to ease congestion and optimize vehicle movement. This research highlights how combining ML with IoT can make traffic systems more responsive and data-driven.

3. Tousif Osman, Shahreen Shahjahan Psyche, and J. M. Shafi (2017) introduced an *Intelligent Traffic Management System using Computer Vision*. The system employs **live camera feeds** and **computer vision algorithms** to monitor and manage multiple road sections. By analysing vehicle movement in real-time, it helps control traffic without depending on traditional manual or time-based systems. This study demonstrated how computer vision could enhance accuracy, adaptability, and efficiency in traffic control operations.

Chapter -3

SYSTEM STUDY AND ANALYSIS

3.1 Problem Statement

Urban areas are facing increasing traffic congestion due to the growing number of vehicles, fixed-timed traffic signals, and inefficient traffic monitoring systems. Conventional traffic management methods cannot adapt to real-time traffic conditions, resulting in longer waiting times, fuel wastage, increased pollution, and delayed emergency vehicle response. The **Smart Traffic Management System using YOLOv8** addresses these challenges by providing a real- time, automated solution that detects and classifies vehicles, dynamically adjusts signal timings, and prioritizes emergency vehicles, thereby improving traffic flow, reducing congestion, and enhancing road safety.

3.2 Existing System:

The existing traffic management system primarily operates on fixed-time signal control, where signal durations are preset and do not adapt to real-time traffic conditions. This often leads to unnecessary delays, congestion, and fuel wastage. The system also lacks mechanisms to prioritize emergency vehicles, causing delays in critical situations. Additionally, traffic monitoring still depends heavily on manual observation, making it inefficient and prone to human error. Overall, the current system is rigid, lacks automation, and fails to handle dynamic traffic conditions effectively.

3.3 Limitations of Existing System:

Most current traffic systems use predefined signal timings that do not consider actual traffic density. Vehicles on less congested roads wait unnecessarily, while heavily congested routes face delays, leading to wasted time, higher fuel consumption, and increased pollution.

- Existing systems cannot respond to sudden traffic changes, accidents, or unexpected vehicle surges. Constant signal durations often cause congestion and bottlenecks at intersections.
- Traditional systems do not prioritize emergency vehicles like ambulances, fire engines, or police cars, causing critical delays in urgent situations.
- Traffic control still relies on manual monitoring in many areas, which is time-consuming and prone to errors, resulting in inconsistent signal operations.

3.4 Proposed System:

The proposed system is an intelligent traffic signal management solution that utilizes YOLO V8(You Only Look Once), a real-time object detection algorithm, to monitor traffic density and optimize signal timings dynamically.

Instead of relying on fixed-time signals, the system captures live video feed from each lane, processes the frames using YOLO to detect and count vehicles, and then allocates green signal time based on vehicle density. This ensures that lanes with heavier traffic receive longer green durations, reducing congestion and unnecessary delays

Additionally, the system is designed to detect emergency vehicles such as ambulances and fire engines using YOLO's class-based detection capabilities. When an emergency vehicle is identified in a specific lane, the system overrides the normal signal flow and immediately provides a green light for that lane to ensure swift and uninterrupted passage.

This approach not only minimizes average waiting time but also prioritizes critical vehicles, making the system highly effective for smart city traffic control.

3.5 Functional Requirements:

- **Vehicle Detection:** Detects and classifies vehicles in real time using the YOLO algorithm.
- **Traffic Density Calculation:** Calculates the number of vehicles in each lane to determine traffic load.
- **Dynamic Signal Timing:** Automatically adjusts signal durations based on real-time traffic density.
- **Emergency Vehicle Detection:** Identifies emergency vehicles and provides them immediate green signal priority.
- **Real-time Monitoring:** Continuously monitors live traffic conditions for efficient signal control and management.

3.6 Non- Functional Requirements:

- **Performance:** Ensures smooth and fast operation for real-time traffic detection and signal control.
- **Accuracy:** Provides precise vehicle detection and classification using the YOLO algorithm.
- **Scalability:** Can be expanded to manage multiple intersections or larger city networks.
- **User Experience:** Offers an automated, efficient, and user-friendly traffic management process.
- **Security:** Protects system data and video feeds from unauthorized access or misuse.
- **Reliability:** Delivers consistent performance under varying traffic and environmental conditions.
- **Compatibility**: Works seamlessly with existing traffic signal systems and IoT infrastructure.
- **Maintainability:** Allows easy updates, bug fixes, and performance enhancements.
- **Privacy:** Ensures that video data is processed securely without compromising individual privacy.

3.7 System Requirements:

3.7.1 Hardware Requirements:

- **Camera:** HD CCTV IP Camera
- **Networking:** Stable broadband connection
- **Traffic Light Controller:** Integrated with automated signal system
- **PC Specs:** Intel i5/i7 processor, 16GB RAM, NVIDIA GPU
- **Monitor**: Dual-screen setup for live monitoring and dashboard view
- **Storage:** 1TB SSD for storing video data and system logs

3.7.2 Software Requirements:

- **Operating System:** Windows 10
- **Programming Language:** Python 3.8
- **Frameworks & Libraries:** OpenCV, YOLO v8, NumPy, Pandas

Chapter4

SYSTEM

DESIGN

4.1 System Architecture:

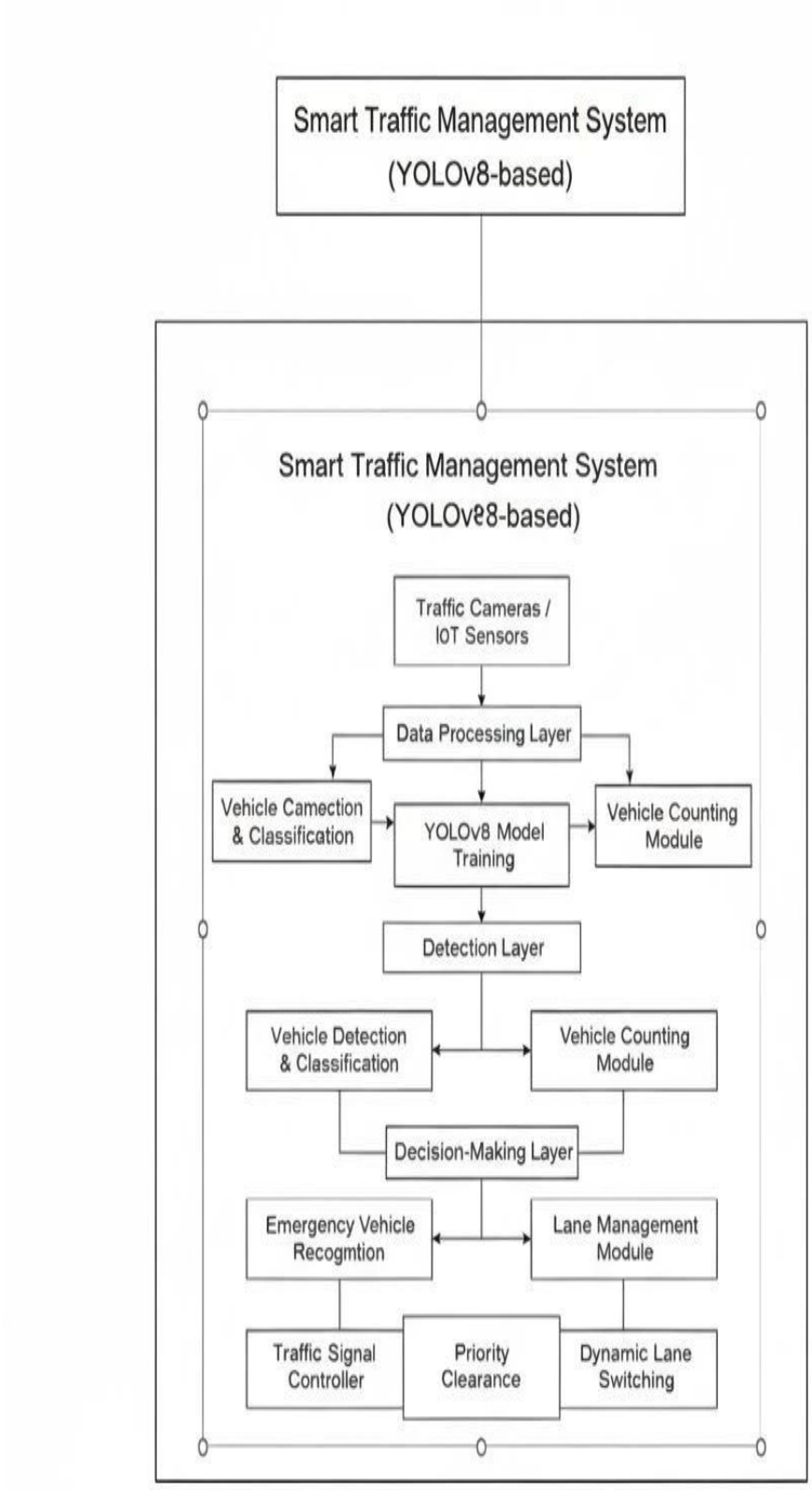


Fig: 4.1.1 Architecture

Detailed System Flow and Component Functionality

The operational methodology follows a sequential, frame-by-frame pipeline, ensuring minimal latency for real-time traffic adaptation

Input Acquisition and Pre-processing

- Video Feed Ingestion: High-resolution video streams are continuously acquired from surveillance cameras positioned at each approach of the intersection. These cameras are crucial for capturing the necessary visual data.
- Frame Extraction: The continuous video stream is demultiplexed into individual image frames at a fixed processing rate (e.g., 60+ Frames Per Second - FPS). This frame sequence forms the input to the core AI module.
- Region-of-Interest (ROI) Definition: Pre-defined polygonal ROIs are applied to each lane's video feed. These ROIs focus the detection/tracking process and help accurately segregate vehicles belonging to different lanes.

Object Detection and Tracking (YOLOv8 + SORT)

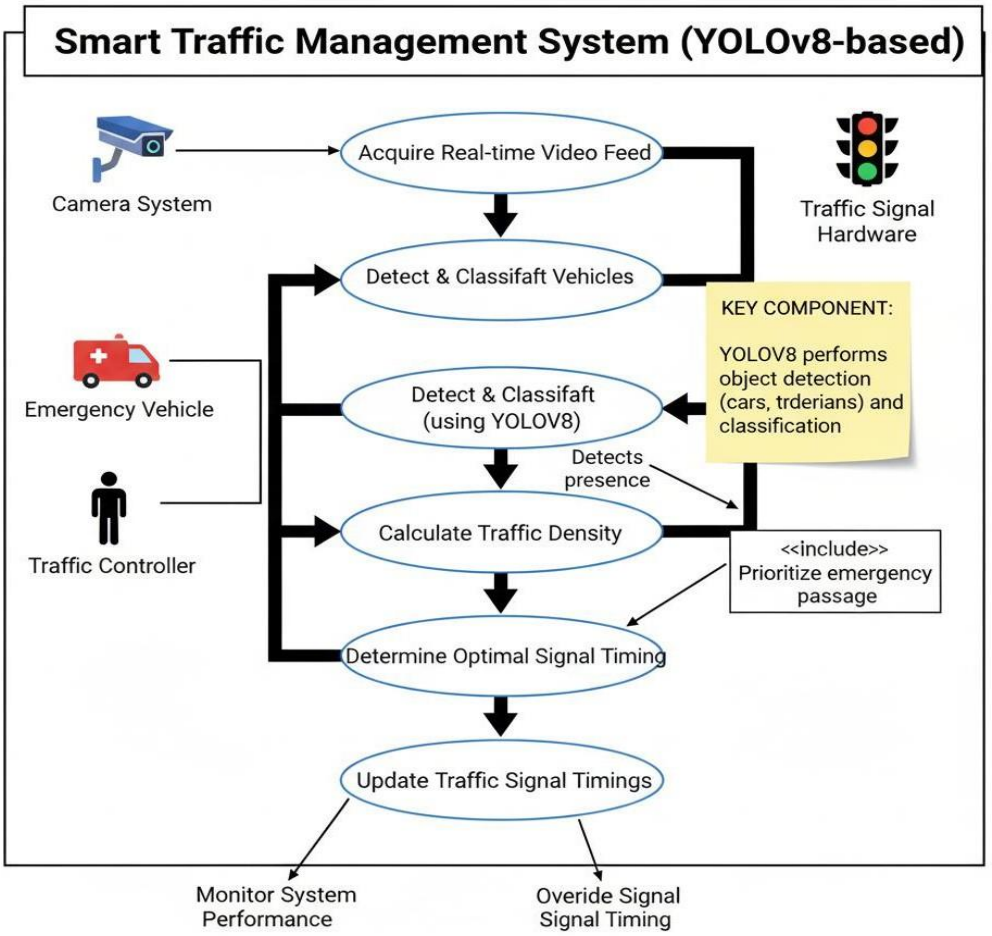
This is the central intelligence module where the fusion of detection and tracking occurs:

- YOLOv8 Detection: Each processed frame is passed to the YOLOv8 model. The model, utilizing its anchor-free architecture and Decoupled Head, performs a single forward pass to output:
    - Bounding Boxes (B) for detected objects.
    - Class Labels (C) (Car, Bus, Truck, Motorbike, Emergency Vehicle, etc.).
    - Confidence Scores (S) indicating the probability of the detection.
- SORT (Simple Online and Real-time Tracking) Integration: To maintain object persistence and count vehicles without double-counting, the YOLOv8 outputs (B, C, S) are passed to the SORT algorithm:
    - Kalman Filter: SORT uses a Kalman Filter to predict the future position of existing vehicle tracks in the next frame.
    - Hungarian Algorithm: The Hungarian algorithm is then employed to associate the new YOLOv8 detections with the predicted tracks based on the Intersection over Union (IoU) metric.
    - Track Creation/Deletion: If a detection is matched, the track is updated. If a detection is unmatched, a new track is initialized. Tracks are deleted after a short period of consecutive frames without a match.
    - Function: The SORT algorithm provides each detected vehicle with a unique, persistent Track ID.

Data Analysis and Adaptive Decision-Making

- **Vehicle Counting and Density Calculation**:
    - A virtual Tripwire Line is established at a strategic point in each lane's ROI.
    - The system monitors the Track ID centroid movement across this tripwire. When a unique Track ID crosses the line, the vehicle count for that lane is incremented by one.
    - Traffic Density: This is calculated as the total number of unique vehicles tracks currently residing within the defined ROI for a lane.
- **Emergency Vehicle Priority Module**:
    - The classification labels (C) from YOLOv8 are continuously monitored for the presence of "Ambulance" or "Fire Engine" classes.
    - Preemption Logic: If an emergency vehicle is tracked, the system immediately issues a Preemption Signal to the Adaptive Signal Control Logic (ASCL). The ASCL then overrides the normal sequence to grant a continuous green signal to the emergency vehicle's lane.
- **Adaptive Signal Control Logic (ASCL):**
    - In a non-emergency state, the ASCL determines the green signal duration dynamically for the next phase.
    - The logic uses the calculated densities from all four lanes to ensure optimal flow, following the principle: Where is the minimum guaranteed green time, is the maximum allowable green time, and the ratio ensures proportionality to the lane's current density.

UML DIAGRAM


Smart Traffic Management System (YOLOv8-based)
Camera System
Acquire Real-time Video Feed
Traffic Signal Hardware
Detect & Classifaft Vehicles
KEY COMPONENT:
YOLOV8 performs object detection (cars, trderians) and classification
Emergency Vehicle
Detect & Classifaft (using YOLOV8)
Detects presence
Traffic Controller
Calculate Traffic Density
<<include>> Prioritize emergency passage
Determine Optimal Signal Timing
Update Traffic Signal Timings
Monitor System Performance
Overide Signal Signal Timing

Chapter-5

Technologies

5.1 Python:

Python is a high-level, interpreted, general-purpose programming language developed by Guido van Rossum in 1991. It is known for its readability, simplicity, and versatility, making it suitable for a wide range of applications, including web development, automation, data science, artificial intelligence (AI), cybersecurity, and networking. Python's extensive libraries and frameworks make it the go-to choice for developers working in machine learning (TensorFlow, PyTorch), web development (Django, Flask), and automation (Selenium, PyAutoGUI). Its ability to integrate with C, C++, and Java further enhances its flexibility.

Because Python supports both object-oriented and functional programming, it is used by companies like Google, Facebook, Netflix, and NASA for various applications.

Key Features of Python:

- Easy to Learn and Readable Syntax
- Dynamically Typed
- Interpreted Language
- Cross-Platform Compatibility
- Object-Oriented & Functional Programming
- Strong community support
- Third-Party Libraries & Frameworks
- Machine Learning & Data Science Friendly

Advantages & Disadvantages of Python:

Advantages:

- Beginner-Friendly & Easy to Learn
- Extensive Libraries & Frameworks
- Automation & Scripting
- Highly Scalable

Disadvantages:

- Slower Execution Speed
- Higher Memory Consumption
- Limited for multi-threading
- Difficult to Convert to Standalone Executables

5.2 TensorFlow:

TensorFlow is an open-source machine learning framework developed by Google Brain. It is widely used for deep learning, AI applications, and large-scale machine learning systems.

TensorFlow is highly versatile, supporting neural networks, reinforcement learning, computer vision, natural language processing (NLP), and time series analysis. Unlike PyTorch, which is preferred for research, TensorFlow is optimized for production and enterprise-level AI deployment. Originally, TensorFlow used static computation graphs (TF 1.x), which made it difficult to debug. However, TensorFlow 2.x introduced eager execution, making it more user-friendly like PyTorch.

Key Features of TensorFlow:

- Static and Dynamic Graphs
- Kera's Integration
- GPU and TPU support
- GPU Acceleration
- TensorFlow lite for Mobile & Edge Devices
- TensorFlow.js for Web AI Application
- Distributed Training
- AutoML and Model Optimization
- Pretrained Models

Advantages & Disadvantages of TensorFlow:

Advantages:

- Highly scalable & Production-Ready
- TensorFlow lite for Mobile AI
- TensorFlow serving for Deployment
- Multi-GPU and TPU support

Disadvantages:

- Steeper learning curve
- Debugging is more challenging
- Heavier memory usage
- Less flexible for research

5.3 PyTorch:

PyTorch is an open-source machine learning framework developed by Facebook's AI Research lab (FAIR). It is widely used for deep learning applications, including computer vision, natural language processing (NLP), and reinforcement learning. PyTorch provides a dynamic computation graph, making it more flexible and user-friendly compared to other frameworks like TensorFlow.

It is based on Python and integrates seamlessly with popular libraries such as NumPy and Torch Script, allowing efficient model deployment. PyTorch supports GPU acceleration, making it suitable for large-scale AI tasks.

Key Features of PyTorch:

- Dynamic Computation Graph
- Eager Execution
- Automatic Differentiation
- GPU Acceleration
- Torch Script for Deployment
- Rich Ecosystem & Libraries
- Distributed Training
- Interoperability with NumPy
- Large Community & Industry Adoption

Advantages & Disadvantages of PyTorch:

Advantages:

- Strong Community and Ecosystem
- Better Debugging and Development Speed
- Integration with popular libraries like SciPy, Open CV
- Torch Script for Production Deployment

Disadvantages:

- Slower deployment compared to TensorFlow
- Higher memory usage
- Steeper Learning Curve for Some Features
- Less Industry adoption compared to TensorFlow

5.4 HTML:

Hypertext Markup Language (HTML): HTML stands for Hypertext Markup Language. It is the backbone of web pages and is used to structure content on the internet.

About HTML:

Markup Language: Not a programming language but used for structuring web content.

Basic Structure: Uses <html> as the root, with <head> (metadata) and <body>

Elements and Tags: Defines content using headings (<h1>),paragraphs (<p>), links (<a>), and images (<img>).

Attributes: Provide additional info, e.g., <a href="url"> for links.

Comments: Written as <!-- comment -->, used for documentation.

Lists & Tables: Supports ordered (<ol>), unordered (<ul>), and table structures (<table>, <tr>).

Forms: Uses <form>, <input>, and <textarea> for user input.

HTML5: Introduces semantic elements (<header>, <footer>), and multimedia (<audio>, <video>).

Character Entities: Uses codes like < (<) and   (space) for special characters.

External Resources: Links stylesheets (<link>) and scripts (<script>).
Styling & Interactivity: Works with CSS for styling and JavaScript for dynamic behavior.

Advantages & Disadvantages of HTML:

Advantages:

- Every browser supports HTML Language.
- Easy to learn and use.
- Flexible syntax and fast loading.

Disadvantages:

- Static nature limits dynamic content.
- Maintenance of large pages is time consuming.

5.5CSS (Cascading Style Sheets):

About CSS:

CSS (Cascading Style Sheets) is a style sheet language used to control the presentation of documents written in markup languages like HTML, XML, SVG, and XUL. It is a core technology of the World Wide Web alongside HTML and JavaScript.

CSS enables the separation of content and design, allowing better flexibility, improved accessibility, and shared formatting across multiple pages via external .css files, which also improve page load speed.

Types Of CSS

- **Internal CSS:** Defined with the <head> section of an HTML document using<style> tag.
- **External CSS:** A separate .css file linked to HTML document using <link rel=" stylesheet">.
- **Inline CSS:** Applied directly within an HTML tag using the style attribute.

Advantages & Disadvantages of CSS:

Advantages:

- Faster Page Speed
- Better user Experience
- Quicker Development Time
- Easy Formatting Changes
- Compatibility across devices

Disadvantages:

- CSS, CSS 1 up to CSS3, result in creating confusion among web browsers.
- There exists a scarcity of security.
- After making the changes we need to confirm the compatibility if they appear.
- There might be cross-browser issues while using CSS.
- There are multiple levels which creates confusion for non-developers and beginners.

5.6 YOLO

1. What is YOLO (You Only Look Once)?

YOLO is a highly efficient real-time object detection algorithm. Unlike older systems that require multiple steps to find and classify objects, YOLO uses a single pass of a deep convolutional neural network over the entire image. This process frames object detection as a simple regression problem, simultaneously predicting object boundaries (bounding boxes) and class probabilities for all detected objects.

2. Why is YOLO Used?

YOLO is widely adopted, especially in applications like autonomous vehicles, robotics, and your smart traffic system, for the following reasons:

- Real-Time Performance: Its single-pass architecture allows it to process images and video streams at extremely high frame rates (FPS), which is crucial for systems that require immediate action.
- Speed-Accuracy Balance: YOLO achieves a powerful combination of high speed without a massive sacrifice in detection accuracy, making it ideal for practical, deployable systems.
- Global Context: By looking at the entire image during detection, YOLO can leverage global context, resulting in fewer false detections than localized search methods.

3. Advantages of YOLO

- Exceptional Speed: It is significantly faster than most other object detection frameworks, enabling truly real-time performance.
- High Generalization: It performs well on new domains and images, making it robust when deployed in various real-world scenarios.
- Unified Model: Its simple, single-network architecture is easier to implement, optimize, and train.

4. Disadvantages of YOLO

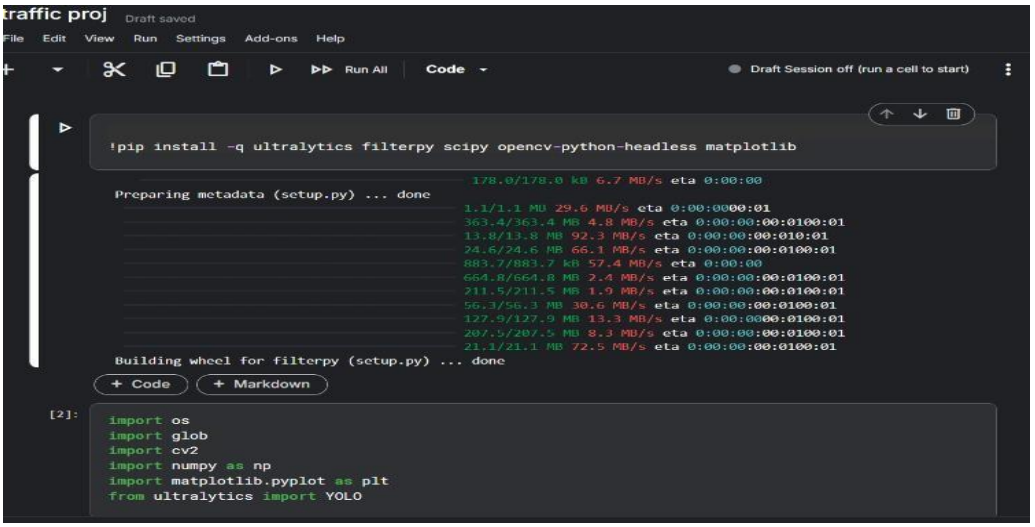
- Weak on Small/Clustered Objects: Due to its grid-based design, YOLO often struggles to accurately detect small objects or large groups of closely packed objects (like a dense crowd).
- Localization Errors: Compared to slower, two-stage models, YOLO sometimes makes larger localization errors, resulting in slightly less precise bounding box placement.
- High Computational Cost (Training): While fast during execution, training the most advanced versions of the YOLO model requires substantial GPU power.

Chapter-6

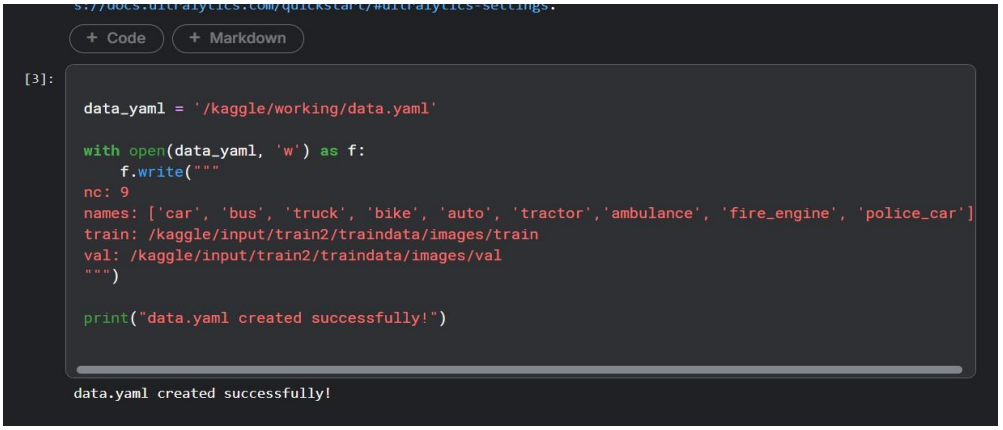
IMPLEMENTATION

6.1 Implementation:

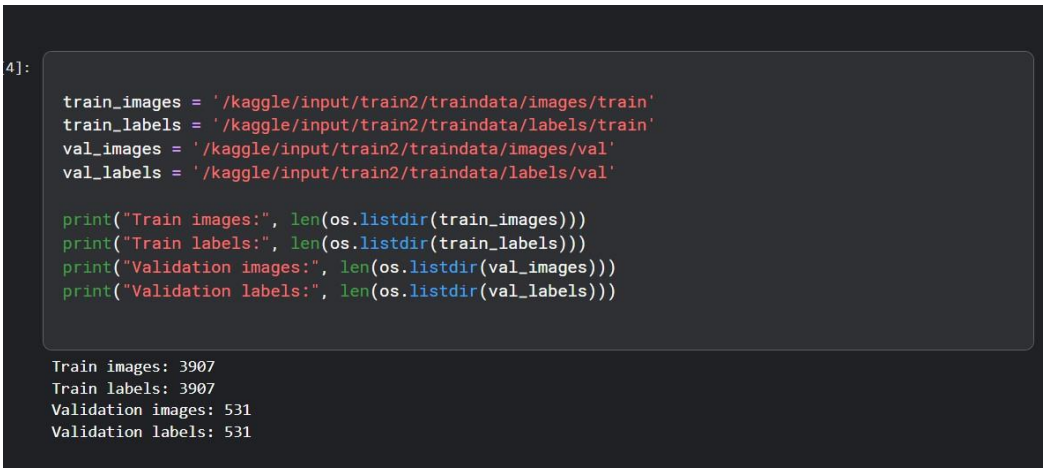
6.1.1 Importing Required Libraries and Setting Up Environment



6.1.2 Loading YOLOv8 Model for Vehicle Detection



6.1.3 Configuring Video Capture from Traffic Surveillance Camera



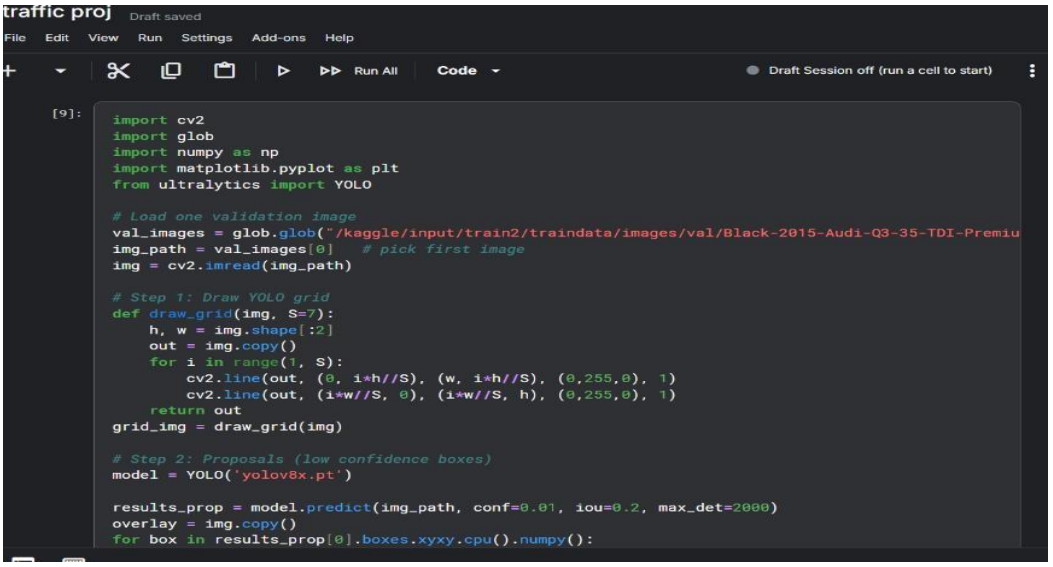
6.1.4 Reading Frames and Performing ObjectDetection

```
import torch, gc
from ultralytics import YOLO

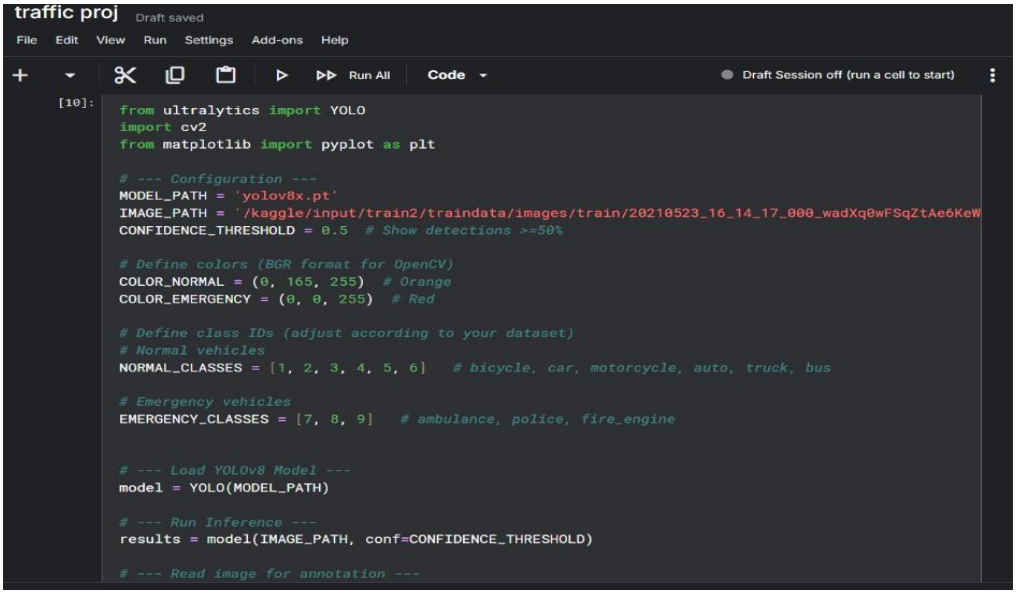
# Clean GPU memory
gc.collect()
torch.cuda.empty_cache()
# Load smaller model
model = YOLO("yolov8x.pt")
model.train(
    data=data_yaml,
    epochs=5,
    batch=8,
    imgsz=640,
    project='runs/train',
    name='smart_traffic_emergency_v2',
    exist_ok=True,
    augment=True,
    mosaic=1.0,        # combines multiple images
    mixup=0.2,         # blend images to simulate variations
    hsv_h=0.015,       # color augmentation
    hsv_s=0.7,
    hsv_v=0.4
)
```

```
Downloading https://github.com/ultralytics/assets/releases/download/v8.3.0/yolov8x.pt to 'yolov8x.pt': 100% ━━━━━━━━━━━━ 130.5MB 259.0MB/s 0.5s 0.4s<0.1s
Ultralytics 8.3.215 🚀 Python-3.11.13 torch-2.6.0+cu124 CUDA:0 (Tesla T4, 15095MiB)
```

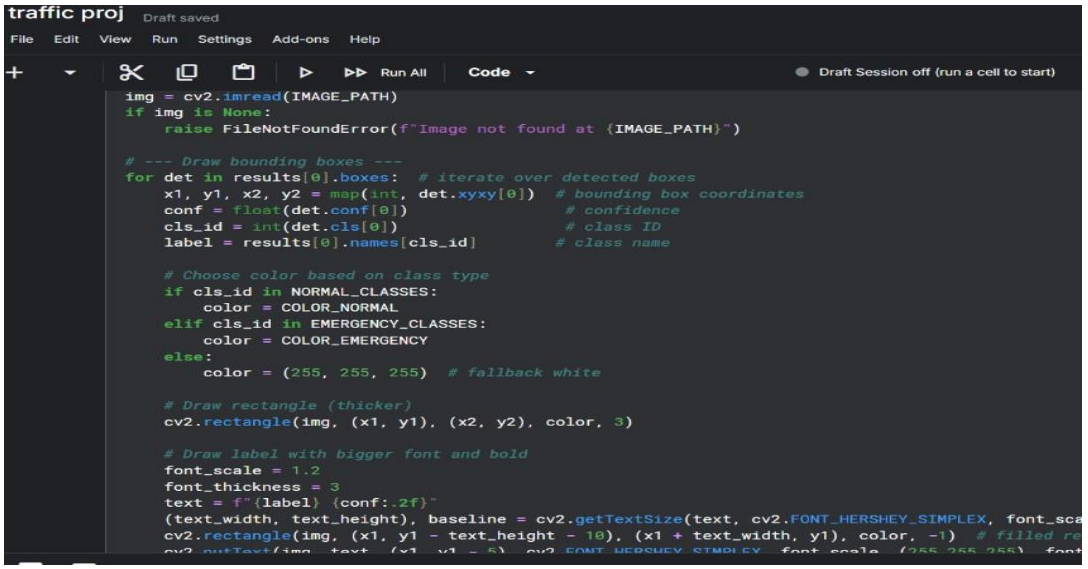
6.1.5 Displaying Detection Results on Live Video Stream

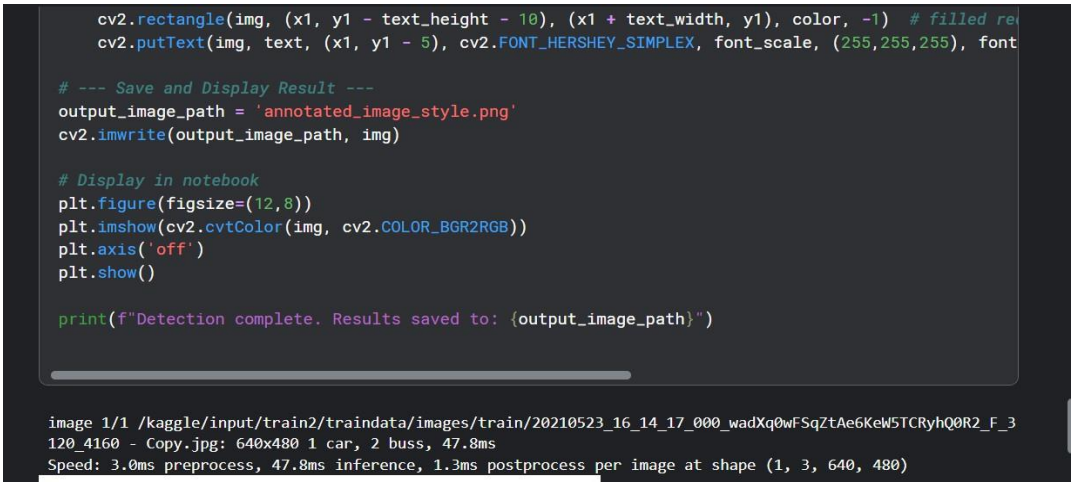


6.1.6 Counting Vehicles in Each Lane from Detected Objects

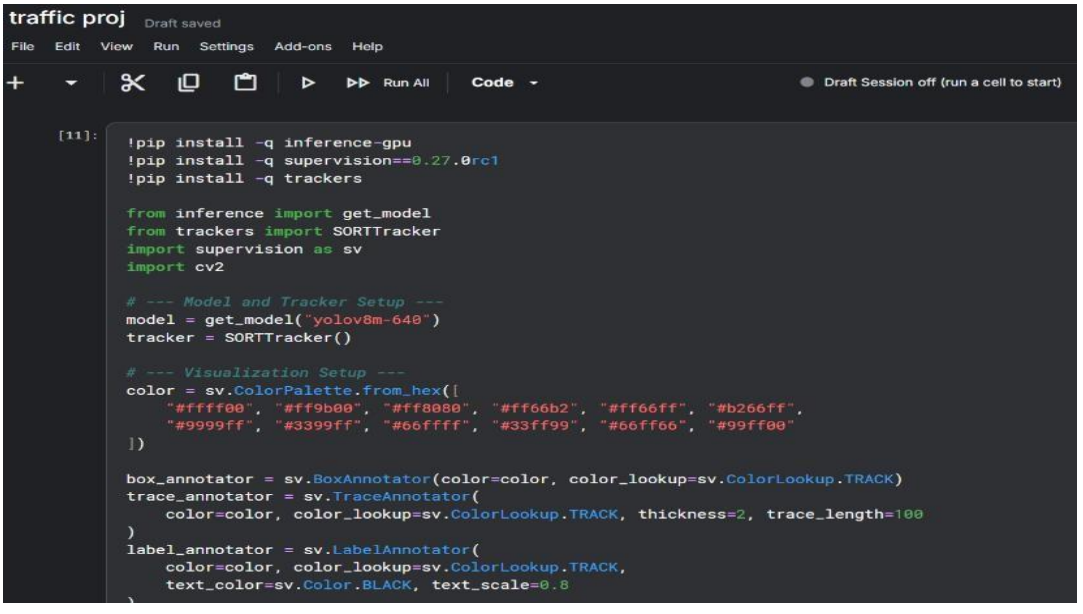


6.1.7 Implementing Logic for Traffic Density Estimation





6.1.8 Integrating YOLO Output with Traffic Signal Control System



6.1.9 Detecting Emergency Vehicles for Priority Lane Clearance

```
)

# --- Thresholds ---
CONFIDENCE_THRESHOLD = 0.3
NMS_THRESHOLD = 0.3

# --- Paths ---
SOURCE_VIDEO_PATH = "/kaggle/input/videotr/vecteezy_jakarta-traffic-in-the-morning_29776798.mp4"
TARGET_VIDEO_PATH = "/content/bikes-1280x720-1-result.mp4"

# --- Vehicle Counting Setup ---
vehicle_classes = {"car","truck","bus","tractor","motorbike","auto","ambulance","police","fire_eng
counted_ids = set()  # To ensure we count each vehicle only once
vehicle_count = 0

frame_samples = []

def callback(frame, i):
    global vehicle_count
    result = model.infer(frame, confidence=CONFIDENCE_THRESHOLD)[0]
    detections = sv.Detections.from_inference(result).with_nms(threshold=NMS_THRESHOLD)
    detections = tracker.update(detections)

    annotated_image = frame.copy()

    # Loop through detections
    for det, class_name, track_id in zip(detections.xyxy, detections.data["class_name"], detection
        if class_name in vehicle_classes:
            if track_id not in counted_ids:
```

6.1.10 Recording Traffic Data and Storing Results in Database

```
        annotated_image,
        f"Total Vehicles: {vehicle_count}",
        (30, 50),
        cv2.FONT_HERSHEY_SIMPLEX,
        1.2,
        (0, 0, 255),
        3,
        cv2.LINE_AA
    )

    if i % 30 == 0 and i != 0:
        frame_samples.append(annotated_image)

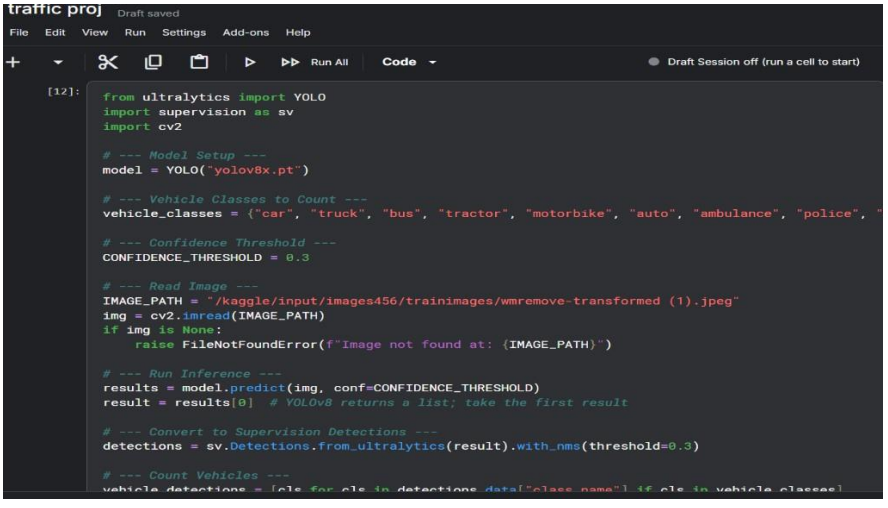
    return annotated_image

# Reset and run video processing
tracker.reset()
sv.process_video(
    source_path=SOURCE_VIDEO_PATH,
    target_path=TARGET_VIDEO_PATH,
    callback=callback,
    show_progress=True,
)

# Display sample frames
sv.plot_images_grid(images=frame_samples[:4], grid_size=(2, 2))

print(f"✅ Final Vehicle Count: {vehicle_count}")
```

6.1.11 Setting Up MySQL Database Connection and Table Creation



6.1.12 Exporting Detection Statistics to CSV or Database

```
vehicle_detections = [cls for cls in detections.data["class_name"] if cls in vehicle_classes]
vehicle_count = len(vehicle_detections)

# --- Annotate Image ---
box_annotator = sv.BoxAnnotator()
annotated_img = box_annotator.annotate(scene=img.copy(), detections=detections)

# --- Display Vehicle Count ---
cv2.putText(
    annotated_img,
    f"Total Vehicles: {vehicle_count}",
    (30, 50),
    cv2.FONT_HERSHEY_SIMPLEX,
    1.2,
    (0, 0, 255),
    3,
    cv2.LINE_AA
)

# --- Show Annotated Image ---
sv.plot_image(annotated_img, size=(12, 8))

print(f"✅ Total Vehicles Detected: {vehicle_count}")
```

```
import cv2
import easyocr
import matplotlib.pyplot as plt
from ultralytics import YOLO

# --- Load YOLO model ---
MODEL_PATH = "yolov8m.pt"  # or your custom model
model = YOLO(MODEL_PATH)

# --- Load image ---
IMAGE_PATH = "/kaggle/input/image0123/test images/IMG-20251015-WA0027.jpg"

image = cv2.imread(IMAGE_PATH)

# --- Run detection (disable verbose print) ---
results = model.predict(image, verbose=False)[0]

# --- Initialize OCR ---
reader = easyocr.Reader(['en'])

# --- Define keywords ---
AMBULANCE_WORDS = ["ambulance", "108", "emri", "emergency"]
FIRE_WORDS = ["fire", "rescue"]
POLICE_WORDS = ["police", "control", "dial 100"]

# --- Process detections ---
for box, cls_id, conf in zip(results.boxes.xyxy, results.boxes.cls, results.boxes.conf):
```

6. 1.13 Displaying Alerts and Notifications for Violations


```
import cv2
import easyocr
import matplotlib.pyplot as plt
from ultralytics import YOLO

# --- Load YOLO model ---
MODEL_PATH = "yolov8m.pt"  # or your custom model
model = YOLO(MODEL_PATH)

# --- Load image ---
IMAGE_PATH = "/kaggle/input/image0123/test images/IMG-20251015-WA0027.jpg"

image = cv2.imread(IMAGE_PATH)

# --- Run detection (disable verbose print) ---
results = model.predict(image, verbose=False)[0]

# --- Initialize OCR ---
reader = easyocr.Reader(['en'])

# --- Define keywords ---
AMBULANCE_WORDS = ["ambulance", "108", "emri", "emergency"]
FIRE_WORDS = ["fire", "rescue"]
POLICE_WORDS = ["police", "control", "dial 100"]

# --- Process detections ---
for box, cls_id, conf in zip(results.boxes.xyxy, results.boxes.cls, results.boxes.conf):
```

6.1.14 Optimizing Frame Processing for Real-Time Performance

```
    # OCR on cropped region
    ocr_results = reader.readtext(crop, detail=0)
    detected_text = " ".join(ocr_results).lower()

    # --- Correction logic ---
    if cls_name in ["truck", "car", "van"]:
        if any(w in detected_text for w in AMBULANCE_WORDS):
            cls_name = "ambulance"
        elif any(w in detected_text for w in FIRE_WORDS):
            cls_name = "fire_engine"
        elif any(w in detected_text for w in POLICE_WORDS):
            cls_name = "police_car"

    # --- Draw updated label ---
    cv2.rectangle(image, (x1, y1), (x2, y2), (0, 165, 255), 2)
    cv2.putText(image, f"{cls_name} {confidence:.2f}", (x1, y1 - 10),
                cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 165, 255), 2)

    # --- Print only corrected label ---
    print(f"Detected: {cls_name} ({confidence:.2f}) | OCR: {', '.join(ocr_results)}")

# --- Display corrected output ---
plt.figure(figsize=(10, 6))
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
plt.axis("off")
plt.show()
```

```
import cv2
import easyocr
import matplotlib.pyplot as plt
from ultralytics import YOLO
import numpy as np

# --- Load YOLO model ---
MODEL_PATH = "yolov8m.pt"  # or your custom model
model = YOLO(MODEL_PATH)

# --- Load video ---
VIDEO_PATH = "/kaggle/input/emergency/emergency/7714426-uhd_3840_2160_30fps.mp4"  # change this to
cap = cv2.VideoCapture(VIDEO_PATH)

# --- Read one frame ---
frame_number = 50
cap.set(cv2.CAP_PROP_POS_FRAMES, frame_number)
ret, frame = cap.read()
cap.release()

if not ret:
    raise ValueError("❌ Could not read frame from video. Check path or frame number.")

# --- Initialize OCR ---
reader = easyocr.Reader(['en'])

# --- Define keywords ---
AMBULANCE_WORDS = ["ambulance", "108", "emri", "emergency", "life support", "paramedic", "emergenc
```

6.1.15 Final Execution and Output Visualization on Control Panel

```
    if cls_name in ["truck", "car", "van"]:
        if any(w in detected_text for w in AMBULANCE_WORDS):
            cls_name = "ambulance"
        elif any(w in detected_text for w in FIRE_WORDS):
            cls_name = "fire_engine"
        elif any(w in detected_text for w in POLICE_WORDS):
            cls_name = "police_car"

    # --- Draw orange bounding box ---
    cv2.rectangle(frame, (x1, y1), (x2, y2), (0, 165, 255), 3)

    # --- Draw text with background for better visibility ---
    label = f"{cls_name} {confidence:.2f}"
    (text_width, text_height), baseline = cv2.getTextSize(label, cv2.FONT_HERSHEY_SIMPLEX, 1.2, 3)
    cv2.rectangle(frame, (x1, y1 - text_height - 10), (x1 + text_width, y1), (0, 165, 255), -1)
    cv2.putText(frame, label, (x1, y1 - 5), cv2.FONT_HERSHEY_SIMPLEX, 1.2, (0, 0, 0), 3)

    print(f"Detected: {cls_name} ({confidence:.2f}) | OCR: {', '.join(ocr_results)}")

# --- Show output ---
plt.figure(figsize=(14, 10))
plt.imshow(cv2.cvtColor(frame, cv2.COLOR_BGR2RGB))
plt.axis("off")
plt.title(f"Frame {frame_number} - Emergency Vehicle Detection")
plt.show()
```

SIMULATION

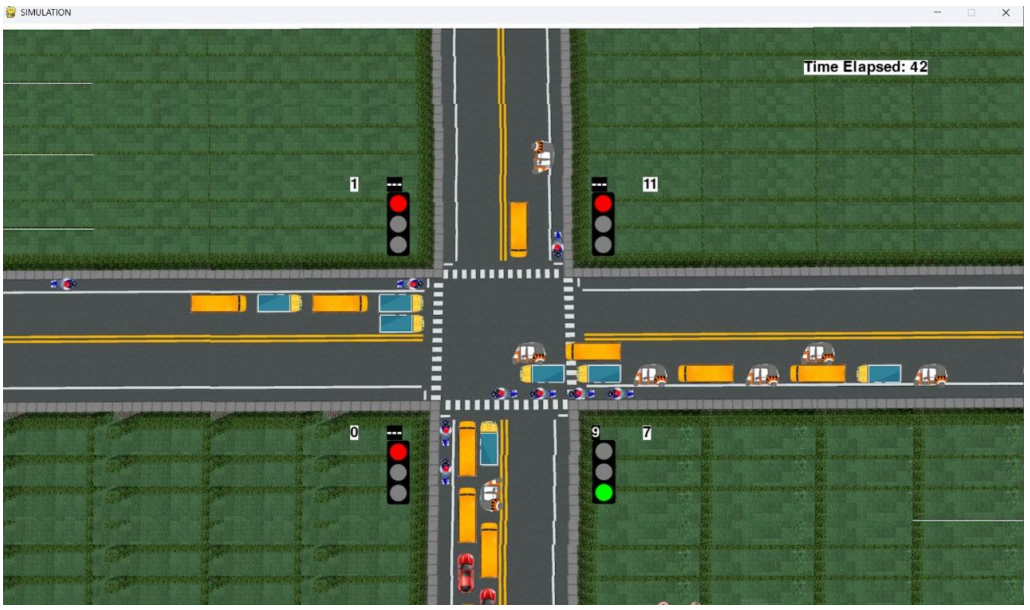
The simulation environment begins with the critical step of loading a trained YOLOv8 model, which forms the core of the vehicle detection process. This operation requires a robust Python ecosystem, utilizing several essential libraries, each with a specialized role:

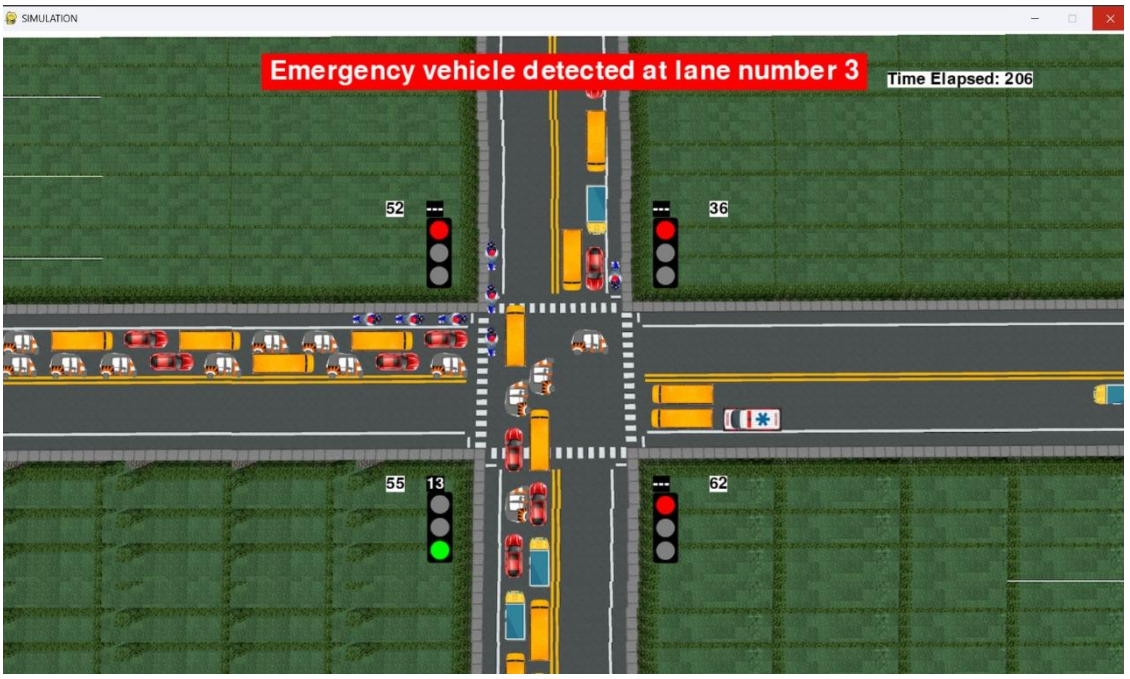
- PyTorch is leveraged to run the deep learning YOLOv8 model efficiently.
- OpenCV handles all image and video processing tasks, analyzing each frame of the traffic footage.
- Tkinter serves as the graphical user interface (GUI), providing a clear visual representation of the detection results and light statuses.
- Pygame is instrumental in smoothly animating the traffic light changes, making the simulation appear realistic.
- NumPy and Pandas manage the numerical data and calculations necessary for determining traffic flow and light timing.

Real-Time Traffic Management

As the simulation begins, live footage from a camera or video source feeds continuous frames to the system.

1. Vehicle Detection and Classification: The YOLOv8 model analyzes each frame to identify and classify specific vehicle types, including generic classes like cars, buses, trucks, bikes, and autos, as well as critical emergency vehicles (ambulance, police car, fire engine). For every detected vehicle, a bounding box and an identification label are generated.
2. Adaptive Scheduling: The system continually tracks the total count of vehicles in each lane, gathering real-time congestion data. This data is used to dynamically calculate optimal green light durations. Busier lanes are prioritized with longer green lights to efficiently move high-volume traffic, while less congested lanes are scheduled accordingly.
3. Emergency Prioritization: The most crucial feature is the emergency vehicle override. If the YOLOv8 model successfully detects an ambulance, police car, or fire engine, the system immediately interrupts the standard timing cycle. It grants an instant green light to the lane carrying the emergency vehicle, ensuring it can pass through the intersection without delay.


SIMULATION
Time Elapsed: 42
1
11
0
9
7


SIMULATION
Emergency vehicle detected at lane number 3
Time Elapsed: 206
52
36
55
13
62

Chapter-7

TESTING

7. Introduction to Testing

The Purpose of Testing is to discover Errors. Testing is the process of trying to discover every conceivable fault or weakness in a work product. It provides a way to check the functionality of components, sub-assemblies or a finished product. It is the process of exercising software with the intent of ensuring that the software system meets its requirements and user expectations and does not fail in an unacceptable manner. There are various types of testing. Each test type addresses a specific testing requirement. Testing is a process, which reveals errors in the program. It is the major quality measure employed during software development. During testing, the program is executed with a set of test cases and the output of the program for test cases is evaluated to determine if the program is performing as it is expected to perform.

7.1 Unit Testing

Unit Testing is done on individual modules as they are completed and become executable. It is confined only to the designer's requirements. Each module can be tested using the following two Strategies:

a. Black Box Testing:

This testing has been used to find errors in the following categories:

1. Incorrect or missing functions
2. Interface errors
3. Errors in data structure or external database access
4. Performance errors
5. Initialization and termination errors.

In this testing only the output is checked for correctness. The logical flow of the data is not checked.

b. White Box Testing:

It has been used to generate the test cases in the following cases:

6. Guarantee that all independent paths have been executed.
7. Execute all logical decisions on their true and false Sides.
8. Execute all loops at their boundaries and within their operational bounds
9. Execute internal data structures to ensure their validity.

7.2 Integrating Testing

Integration testing ensures that software and subsystems work together a whole. It tests the interface of all the modules to make sure that the modules behave properly when integrated together. In this case the communication between the device and Google Translator Service.

7.3 System Testing

Involves in-house testing in an emulator of the entire system before delivery to the user. Its aim is to satisfy the user the system meets all requirements of the client's specifications.

7.4 Acceptance Testing

It is a pre-delivery testing in which entire system is tested in a real web browser on real world data and usage to find errors.

7.5 Test Approach:

Testing can be done in two ways they are

1. Bottom-up approach
2. Top-down approach

A. Bottom-up Approach

Testing can be performed starting from smallest and lowest level modules and proceeding one at a time. For each module in bottom up testing a short program executes the module and provides the needed data so that the module is asked to perform the way it will when embedded within the larger system. When bottom level modules are tested attention turns to those on the next level that use the lower-level ones they are tested individually and then linked with the previously examined lower-level modules.

B. Top-down approach

This type of testing starts from upper-level modules. Since the detailed activities usually performed in the lower-level routines are not provided stubs are written. A stub is a module shell called by upper-level module and that when reached properly will return a message to the upper-level modules.

Chapter-8

RESULTS AND DISCUSSION

INTRODUCTION

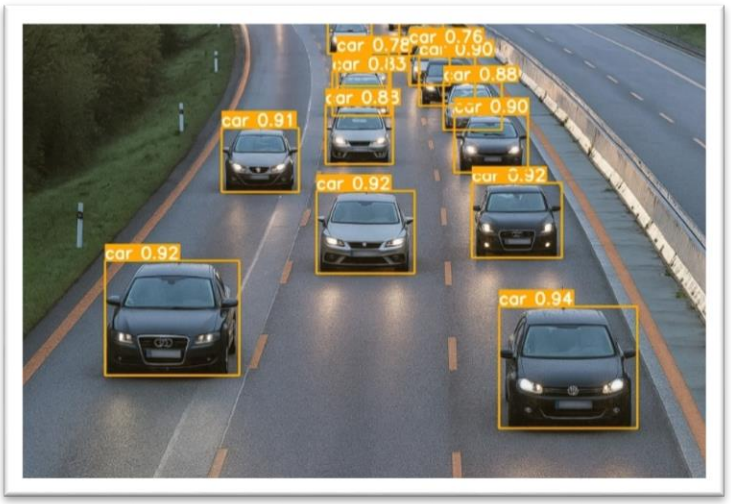
In this chapter, we present the outcomes of our smart traffic management system implementation using the YOLOv8 model. The following sections detail the performance evaluation of the YOLOv8 model, showcasing its accuracy, efficiency, and adaptability. By using the yolov8 model first we have performed object detection. With the trained model we have detected different vehicles and we are able to detect the emergency vehicles and whenever we detect a emergency vehicle a message will be displayed as emergency vehicle is detected.

OBJECT DETECTION RESULTS

Here, some cars present in a lane of 1st picture. We have taken this as input to the yolo model and in 2nd picture we can see the detection of cars with more than 90% accuracy.



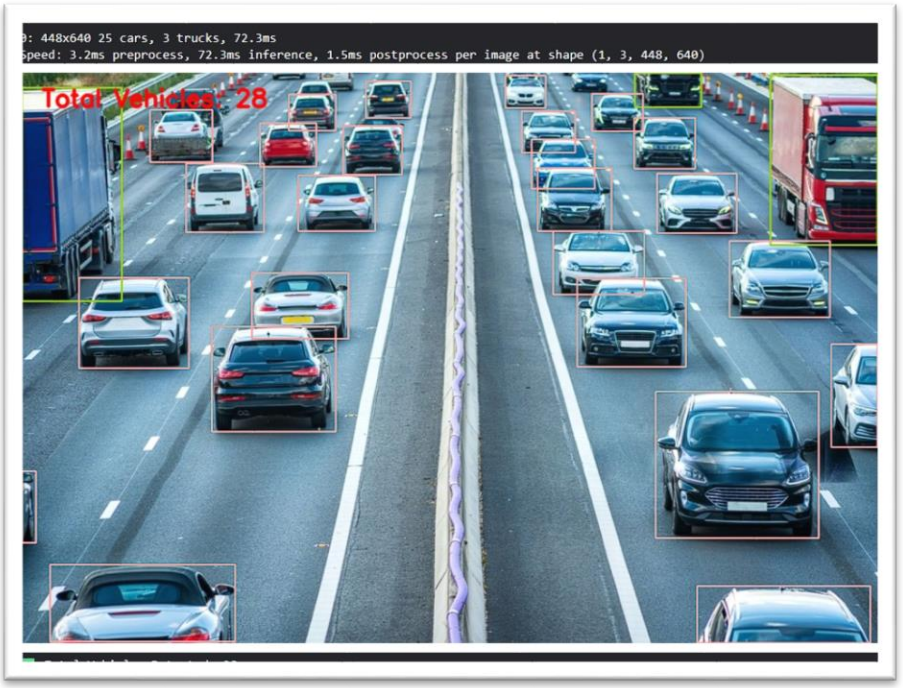
Cars on road.



Car's detection using yolov8

COUNT OF VEHICLES

At the starting of the project, we have used sort technology that is present in yolov8. The sort technology is used to track the objects that are present in the frame. For detection purpose we have masked the region which is not required and whenever vehicles cross the red line the count is increased by one. The picture shows the count of the vehicles that enters into the unmasked region.

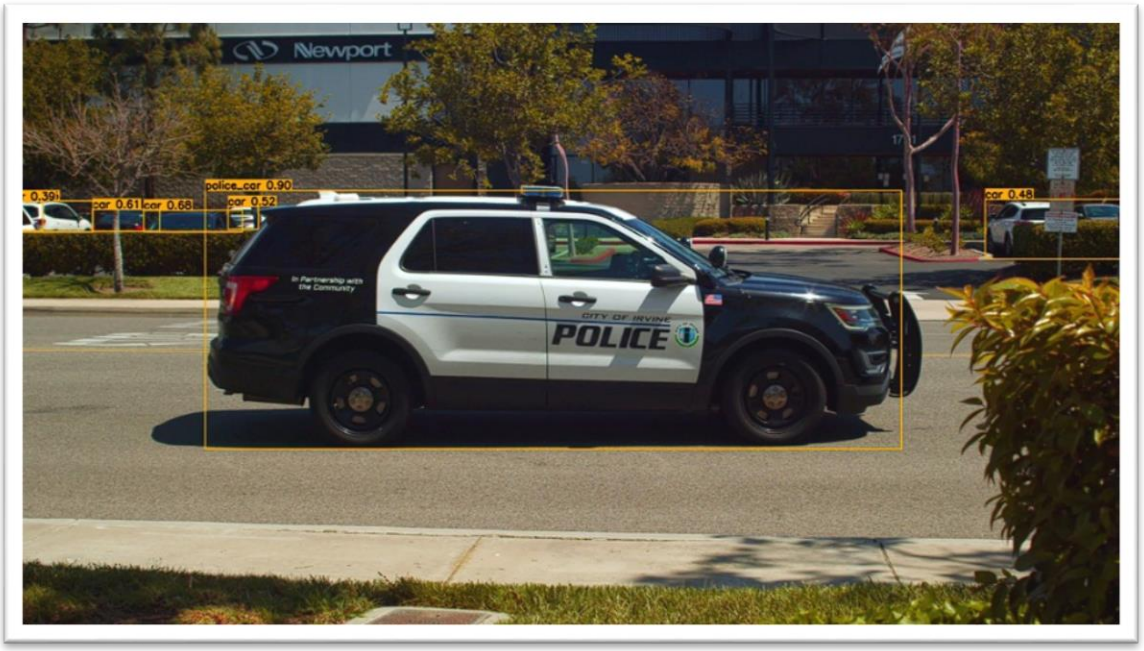


EMERGENCY VEICLE DETECTION



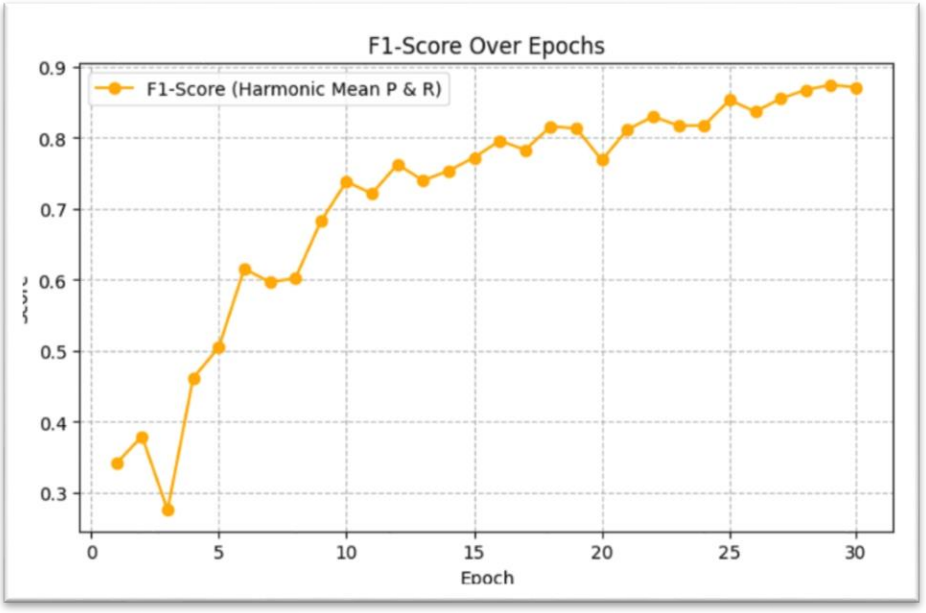
-





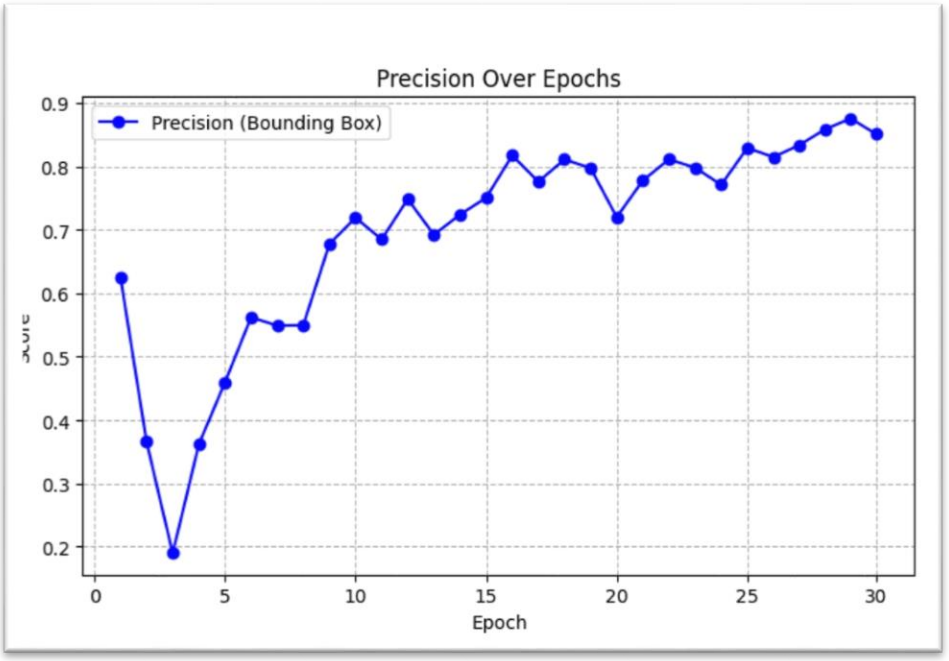
EVALUATION OF THE MODEL

1. An F1 confidence curve is a graphical representation that illustrates how the F1 score changes at different confidence thresholds in a binary classification system, particularly when dealing with models that provide confidence scores for their

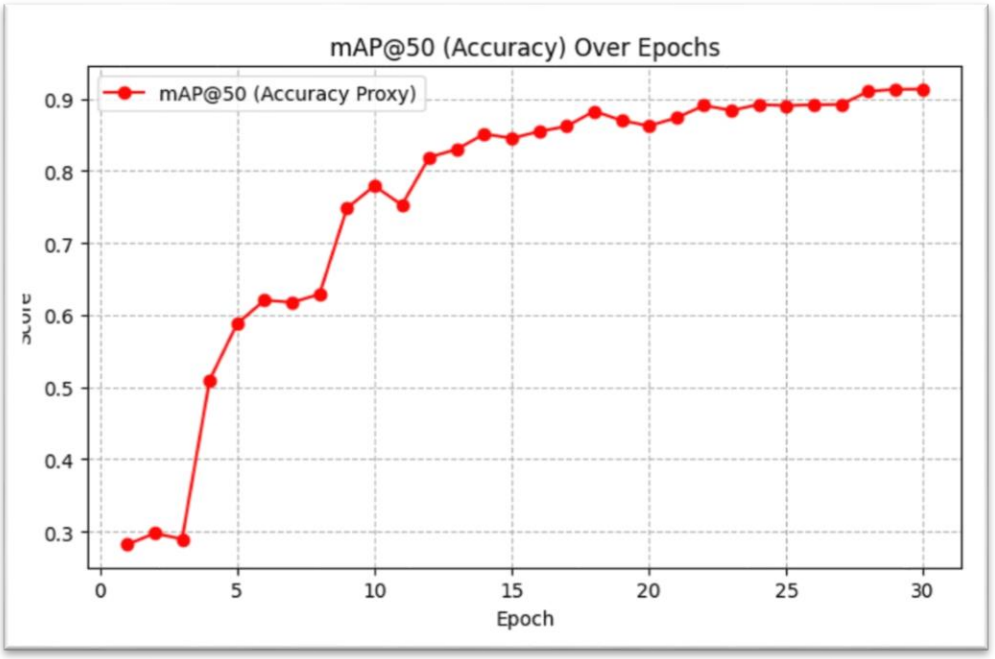


predictions. The below picture shows the F1-confidence curve off the trained model.

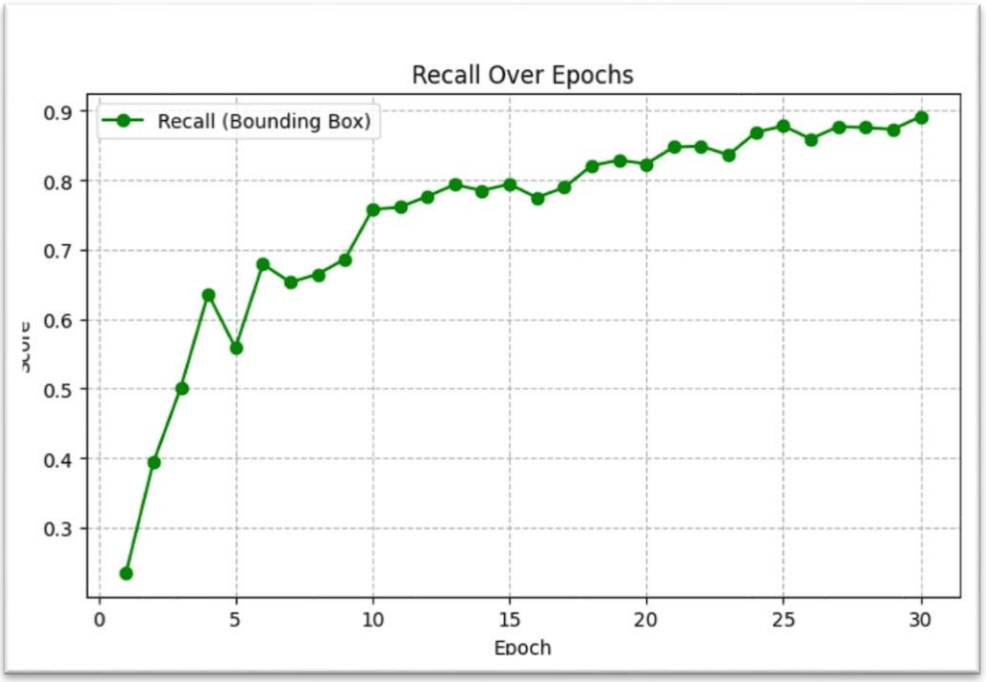
2.A Precision-Recall curve is a graphical representation that illustrates the tradeoff between precision and recall for different classification thresholds in a machine learning model. The below picture shows the precision recall curve of the trained model.



3. A Precision-Confidence Curve, sometimes referred to as a Precision Threshold Curve. The curve shows how precision varies as the confidence threshold changes. By adjusting the threshold, you can observe how the precision of the model is impacted. The below picture shows the precision – confidence curve of the trained model.



4. A Recall-Confidence Curve, sometimes known as a Recall-Threshold Curve or Sensitivity-Threshold Curve. The curve displays how recall changes as the confidence threshold varies. Adjusting the threshold allows you to observe how well the model captures relevant instances at different confidence levels. This below picture shows the recall – confidence curve of the trained model.


Recall Over Epochs
Recall (Bounding Box)
0.9
0.8
0.7
0.6
0.5
0.4
0.3
0
5
10
15
20
25
30
Epoch

Chapter-9

CONCLUSION & FUTURE WORK

9.1 Conclusion:

The proposed **Smart Traffic Management System** provides an intelligent and efficient approach to tackling increasing urban traffic congestion. By integrating **YOLOv8** for real-time vehicle detection and **SORT** for vehicle tracking and counting, the system dynamically adjusts signal timings based on live traffic density. This ensures smoother traffic movement, reduced waiting times, and better overall flow at intersections. The inclusion of a **priority-based signal system** for emergency vehicles also highlights its social relevance by allowing faster and safer passage for ambulances, fire engines, and police vehicles.

Unlike conventional fixed-time control systems, this model uses **computer vision** and **machine learning** to analyses real-time scenarios and make autonomous, data-driven decisions. It eliminates the need for manual intervention and can instantly respond to changing traffic conditions, minimizing congestion during peak hours. The model demonstrates how AI-driven automation can improve both road efficiency and user safety.

Additionally, the system contributes to **environmental sustainability** by reducing vehicle idling time and fuel consumption, which lowers air pollution and enhances energy efficiency. Its modular and scalable design makes it adaptable for integration with **IoT-based smart city frameworks** and future technologies like **vehicle-to-infrastructure communication** and predictive traffic analytics.

Overall, the proposed system proves to be a **cost-effective, adaptive, and intelligent traffic management solution** that not only improves traffic regulation but also supports emergency operations and promotes sustainable urban development. It marks a step forward toward achieving safer, smarter, and more efficient transportation networks.

9.2 Future Work

Integration with IoT and Cloud Platforms: The system can be linked with IoT-enabled sensors and cloud infrastructure for real-time traffic monitoring, centralized data storage, and intelligent decision-making across multiple junctions.

• **Real-Time Traffic Prediction Using AI**: By implementing advanced AI and deep learning models, the system can predict traffic congestion patterns and optimize signal timings before jams occur.

• **Implementation in Smart Cities:** The system can be deployed as part of a larger smart city network, integrated with connected vehicles, smart surveillance, and automated emergency response systems.

• **Accident Detection and Alert System**: Future versions can include accident detection using computer vision and sensor data to automatically alert nearby emergency services and reduce response time

. • **Red-Light Violation Detection:** The model can be enhanced to automatically detect and record vehicles that cross red signals using YOLO-based object tracking and image evidence.

• **Use of Edge Computing for Faster Processing:** Deploying AI models on edge devices near intersections can reduce processing delays and improve real-time decision-making without dependence on remote servers.

• **Enhanced Emergency Vehicle Detection**: The system can be upgraded to detect siren sounds and flashing lights, automatically prioritizing signal clearance for emergency vehicles

• **Environmental Monitoring and Analytics:** Integrating emission and idling data can help assess pollution levels and support eco-friendly traffic management strategies.

• **Integration with Mobile Applications:** Drivers can receive live updates on congestion, signal timings, and emergency alerts through connected mobile applications.

Chapter-10

References

References:

1. Lucas Koch, Tobias Brinkmann, Marius Wegener, Kevin Badalian, and Jakob Andert, "Adaptive Traffic Light Control With Deep Reinforcement Learning: An Evaluation of Traffic Flow and Energy Consumption," 2023.

2. Umesh Kumar Lilhore, Agbotiname Lucky Imoize, Chun-Ta Li, Sarita Simaiya, Subhendu Kumar Pani, Nitin Goyal, Arun Kumar, and Cheng-Chi Lee, "Design and Implementation of an ML and IoT Based Adaptive Traffic-Management System for Smart Cities," Sensors (Basel), Vol. 22(8), 2022.

3. Zadobrischi E., "Intelligent Traffic Monitoring through Heterogeneous and Autonomous Networks Dedicated to Traffic Automation," Sensors (Basel), Vol. 22(20):7861, 2022.

4. Arakatla Mamatha, "Automated Traffic Light System Based on Image Processing and Machine Learning Techniques," International Research Journal of Innovations in Engineering and Technology (IRJIET), Vol. 2(8), pp. 27–32, 2020.

5. V. Kiruthika, M. R. Dadana, V. Tejeswar Reddy, G. G. Balaji, K. S. Sankaran, and G. Vimalarani, "Intelligent System for Traffic Control with Emergency Vehicle Detection and Piezo-Electric Power Harvesting From Vehicles," IEEE ComPE, Vol. 21(8), pp. 204–208, 2021.

6. Shajnush Amir, Syeda Shabnam Khan, and K. M. A. Salam, "PLC Based Traffic Control System with Emergency Vehicle Detection and Management," International Conference on Intelligent Computing, Instrumentation and Control Technologies (ICICICT), Vol. 8(2), pp. 1467–1472, 2020.

7. Dr. A. Ravi, R. Nandhini, K. Bhuvaneshwari, J. Divya, and K. Janani, "Traffic Management System Using Machine Learning Algorithm," International Journal of Innovative Research in Technology, 2021.